Integración de datos

Trabajo Práctico Final

Sergio Enrique Morel Peralta
Clara Patricia Almirón Burgos
Daniel Ramírez
Facultad Politécnica - UNA, 12.04.2026

1. Pipeline ELT: Análisis de Accidentes de Tráfico en Brasil

Este trabajo implementa un pipeline ELT completo que integra datos de accidentes de tráfico registrados por la Policía Rodoviária Federal de Brasil (DATATRAN/PRF) con datos climáticos históricos de la API Open-Meteo ERA5. El objetivo es construir un modelo dimensional analítico que permita explorar la relación entre condiciones climáticas y accidentalidad vial en rutas federales brasileñas durante 2026.

La arquitectura del pipeline sigue la secuencia: **MySQL → Airbyte → MotherDuck → dbt → Metabase**, orquestada con **Prefect**.

Repositorio: https://github.com/SEMP/integracion-de-datos-final

1.1. Introducción

1.1.1. Problemática y motivación

Los accidentes de tráfico representan una de las principales causas de mortalidad en Brasil. La Policía Rodoviária Federal registra sistemáticamente cada accidente ocurrido en rutas federales, incluyendo coordenadas GPS, condiciones

meteorológicas según el agente, y datos de víctimas. Sin embargo, la condición climática registrada por el agente (`condicao_metereologica`) es subjetiva y limitada. Enriquecer este dataset con datos climáticos objetivos —precipitación real, temperatura, cobertura de nubes— permite análisis más precisos de la influencia del clima en la accidentalidad.

1.1.2. Fuentes de datos

1.1.2.1. Fuente 1: DATATRAN PRF 2026

El dataset proviene del portal de datos abiertos del gobierno brasileño [1] y contiene todos los accidentes registrados por la PRF en rutas federales durante el año 2026.

Característica	Valor
Registros	11.380 accidentes
Período	Enero – Abril 2026
Encoding	ISO-8859-1 (Latin-1)
Separador	Punto y coma (`;`)
Fin de línea	CRLF (Windows)
Coordenadas	Latitud y longitud GPS por accidente
Decimales	Coma como separador (ej: `-7,291548`)

Las 30 columnas del dataset cubren localización (UF, municipio, BR, km, lat/lon), causa y tipo de accidente, condición meteorológica según el agente PRF, fase del día, características de la vía, y conteos de víctimas (muertos, heridos leves, heridos graves, ilesos).

Un campo de atención especial es `tracado_via`, que puede contener múltiples valores separados por punto y coma dentro del mismo campo (ej: `"Aclive;Reta"`), lo que requiere que las herramientas de carga respeten el entrecomillado del CSV.

1.1.2.2. Fuente 2: Open-Meteo ERA5 (clima histórico)

Open-Meteo [2] provee datos de reanálisis ERA5 con resolución horaria para cualquier coordenada geográfica. El join con DATATRAN se realiza mediante `data_inversa` + `horario` + `latitude` + `longitude` de cada accidente.

Variable ERA5	Relevancia vial
`precipitation`	★★★ — Precipitación mm/h
`weather_code` (WMO)	★★★ — Condición climática estandarizada
`temperature_2m`	★★ — Temperatura ambiente
`relative_humidity_2m`	★★ — Humedad relativa

Variable ERA5	Relevancia vial
`dew_point_2m`	★★ — Punto de rocío (riesgo de escarcha)
`is_day`	★★ — Flag día/noche
`cloud_cover_low`	★ — Nubes bajas (proxy de niebla)
`wind_speed_10m`	★ — Velocidad del viento
`wind_gusts_10m`	★ — Ráfagas de viento
`shortwave_radiation`	★ — Radiación solar (encandilamiento)

1.1.2.3. Decisión: Open-Meteo ERA5 vs OpenWeather One Call 3.0

Se evaluaron dos APIs para datos climáticos históricos. Ambas se probaron contra 5 accidentes reales del dataset, obteniéndose 5/5 respuestas exitosas en las dos. La tabla siguiente resume los factores determinantes:

Criterio	Open-Meteo ERA5	OpenWeather 3.0
Costo	Gratuito, sin API key	Suscripción paga
`visibility`	No disponible	No disponible
`wind_gusts`	Disponible	`None` en todos los casos
Nubes bajas	`cloud_cover_low`	No disponible
Radiación solar	`shortwave_radiation`	No disponible
Fase del día	Flag `is_day`	Calculado de sunrise/sunset
Clasificación WMO	Estándar internacional	Etiquetas propias

Se eligió **Open-Meteo ERA5** por ser gratuito y proveer mayor cantidad de variables útiles. La visibilidad (`visibility`) no está disponible en ninguna de las dos APIs para datos ERA5 históricos; se usa `condicao_metereologica` de la PRF como proxy.

1.2. Paso 1: Pre-procesamiento del CSV

Antes de levantar MySQL, el CSV original debe convertirse a UTF-8. Esta conversión es necesaria porque:

- MySQL 8.0 con `utf8mb4` rechaza datos Latin-1 si el cliente no indica el charset correcto.
- Los campos con acentos (`Céu Claro`, `carroçável`) quedan corruptos si se carga el archivo sin conversión explícita.
- Normalizar CRLF a LF simplifica el `LOAD DATA INFILE`.

1.2.1. Script de conversión

El script `workspaces/scripts/convert_csv_to_utf8.py` realiza la conversión usando únicamente la biblioteca estándar de Python:

```python
import csv

with (
    open(input_path, encoding="latin-1", newline="") as src,
    open(output_path, "w", encoding="utf-8", newline="\n") as dst,
):
    reader = csv.reader(src, delimiter=";", quotechar='"')
    writer = csv.writer(dst, delimiter=";", quotechar='"',
                        quoting=csv.QUOTE_MINIMAL, lineterminator="\n")
    for row in reader:
        writer.writerow([field.strip() for field in row])
```

El uso del módulo `csv` con `quotechar='"'` es crítico: garantiza que el campo `tracado_via` —que contiene ; internos como `"Aclive;Reta"`— sea leído y reescrito como un único campo entrecomillado, sin romper la estructura del CSV.

1.2.2. Ejecución

```python
python3 workspaces/scripts/convert_csv_to_utf8.py
# Salida esperada:
# Leyendo  : .../datatran2026.csv
# Escribiendo: .../datatran2026_utf8.csv
# Filas escritas (incluye encabezado): 11381
```

El archivo resultante `datatran2026_utf8.csv` queda en el mismo directorio que el original y es el que se monta en el contenedor MySQL.

1.3. Paso 2: Base de datos MySQL con Docker

1.3.1. Contenedor Docker

Se levanta MySQL 8.0 junto a phpMyAdmin y Metabase mediante `docker-compose.yaml` en `workspaces/containers/`. Las credenciales se leen desde un archivo `.env` (no versionado); el archivo `example.env` sirve de plantilla.

```yaml
services:
  mia-mysql:
    image: mysql:8.0
    command: >
      --default-authentication-plugin=mysql_native_password
      --local-infile=1
      --secure-file-priv=/csv
    volumes:
      - ./initdb:/docker-entrypoint-initdb.d:ro
      - ${CSV_DIR}:/csv
```

La opción `--default-authentication-plugin=mysql_native_password` garantiza compatibilidad con el conector de Airbyte. La directiva `--secure-file-priv=/csv`

restringe `LOAD DATA INFILE` al directorio `/csv`, donde se monta `${CSV_DIR}` (la carpeta local con el CSV convertido).

1.3.2. Schema: todo TEXT

La tabla `accidentes_raw` define todos sus campos como `TEXT`. Esta decisión evita cualquier fallo de conversión de tipos durante la ingesta y delega la lógica de tipado a dbt, donde es versionable y testeable:

Campo	Tipo MySQL	Motivo
`km`, `latitude`, `longitude`	TEXT	Decimal con coma: `-7,291548`
`classificacao_acidente`	TEXT	Contiene el valor literal `NA`
`tracado_via`	TEXT	Valores múltiples con `;` interno
`data_inversa`	TEXT	String — dbt castea a DATE
`horario`	TEXT	String — dbt castea a TIME
`id`, `mortos`, etc.	TEXT	Enteros — dbt castea a INT/BIGINT

```
CREATE DATABASE IF NOT EXISTS datatran
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE datatran;

CREATE TABLE accidentes_raw (
    id TEXT, data_inversa TEXT, dia_semana TEXT,
    horario TEXT, uf TEXT, br TEXT, km TEXT,
    municipio TEXT, causa_acidente TEXT, tipo_acidente TEXT,
    classificacao_acidente TEXT, fase_dia TEXT, sentido_via TEXT,
    condicao_metereologica TEXT, tipo_pista TEXT, tracado_via TEXT,
    uso_solo TEXT, pessoas TEXT, mortos TEXT, feridos_leves TEXT,
    feridos_graves TEXT, ilesos TEXT, ignorados TEXT, feridos TEXT,
    veiculos TEXT, latitude TEXT, longitude TEXT,
    regional TEXT, delegacia TEXT, uop TEXT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

1.3.3. Carga con LOAD DATA INFILE

```
LOAD DATA INFILE '/csv/datatran2026_utf8.csv'
INTO TABLE accidentes_raw
CHARACTER SET utf8mb4
FIELDS
    TERMINATED BY ';'
    ENCLOSED BY '"'
LINES
    TERMINATED BY '\n'
IGNORE 1 LINES;
```

`ENCLOSED BY '"'` es la cláusula crítica: sin ella, los valores de `tracado_via` que contienen `;` interno (como `"Aclive;Reta"`) se interpretarían como dos campos separados, corrompiendo todas las columnas a partir de esa posición.

1.3.4. Verificación

```
docker exec -it mia-mysql mysql -u airbyte -pairbyte datatran \
  -e "SELECT COUNT(*) AS total FROM accidentes_raw;"
-- Resultado esperado: 11380

docker exec -it mia-mysql mysql -u airbyte -pairbyte datatran \
  -e "SELECT tracado_via FROM accidentes_raw
      WHERE tracado_via LIKE '%;%' LIMIT 3;"
-- Resultado: Aclive;Reta | Declive;Curva | Reta;Em Obras
```

1.4. Paso 3: Configurar Airbyte

1.4.1. Source: MySQL (DATATRAN)

Campo	Valor
Nombre	`MySQL_Datatran`
Host	`10.147.20.165` (IP ZeroTier — estable, no cambia)
Port	`3306`
Base de datos	`tf-datatran`
Usuario	`airbyte`
Encryption	required
SSH Tunnel	No Tunnel
Update Method	Scan Changes with User Defined Cursor

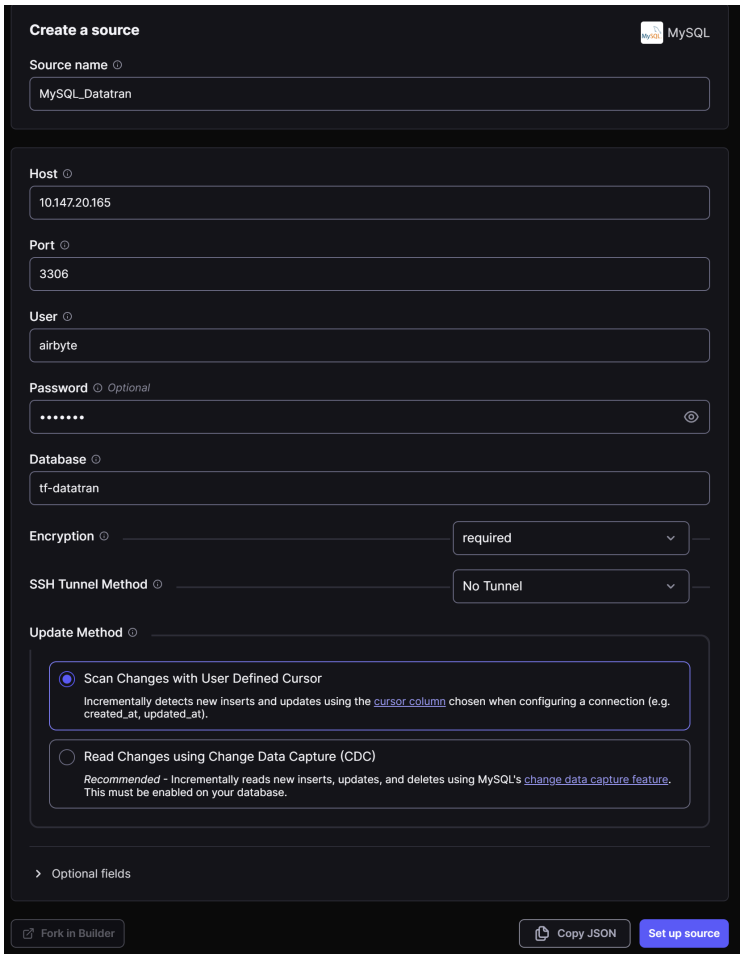

Figura 1: Configuración del source MySQL en Airbyte

1.4.2. Source: Open-Meteo ERA5 (clima)

Campo	Valor
Tipo	Custom connector / script de extracción
Endpoint	`https://archive-api.open-meteo.com/v1/archive`
Parámetros	`latitude`, `longitude`, `start_date`, `end_date`, `hourly`
Sin API key	Gratuito

1.4.3. Destination: MotherDuck

Campo	Valor
Nombre	`MotherDuck_datatran`
Destination DB	`md:airbyte_trabajo`
Schema Name	`datatran`

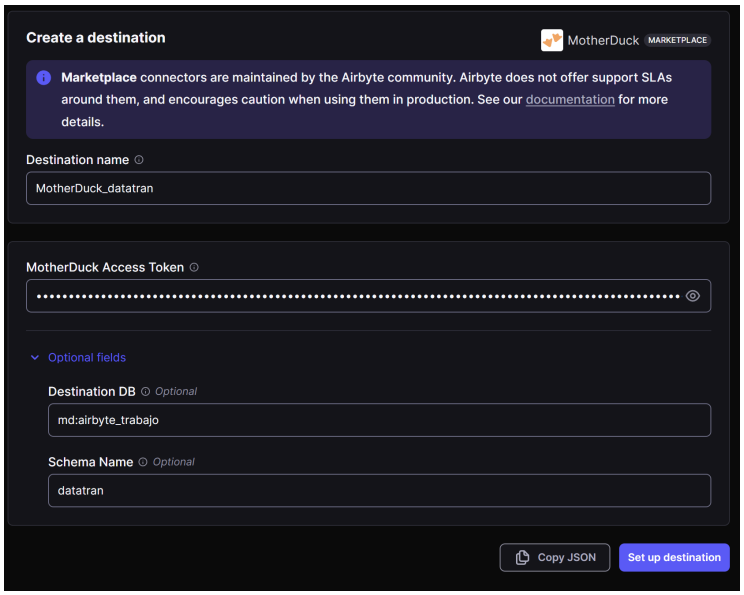

Figura 2: Configuración del destination MotherDuck en Airbyte

1.4.4. Connection: sync mode

Se seleccionó **Full Refresh | Overwrite** para la tabla `accidentes_raw`. El dataset DATATRAN 2026 es estático (no recibe actualizaciones incrementales) y no dispone de columnas confiables de timestamp de modificación (`created_at`, `updated_at`) que habilitarían un modo incremental. La estrategia Overwrite garantiza que cada sync reemplaza completamente el contenido de la tabla destino, manteniendo consistencia sin riesgo de duplicados.

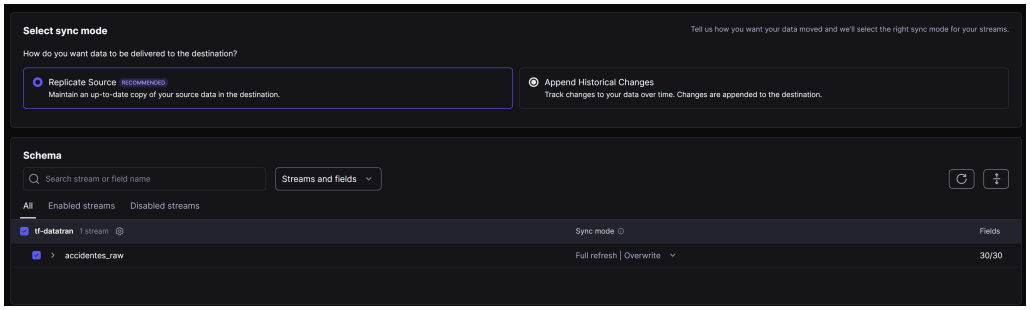

Figura 3: Selección de sync mode: Full Refresh | Overwrite sobre `accidentes_raw` (30/30 campos)

El paso siguiente configura los metadatos de la connection: nombre, tipo de schedule y namespace de destino. Se eligió **Destination-defined** como namespace para que los datos queden en el schema `datatran` configurado en el destination, y **Manual** como schedule dado que la orquestación la maneja Prefect.

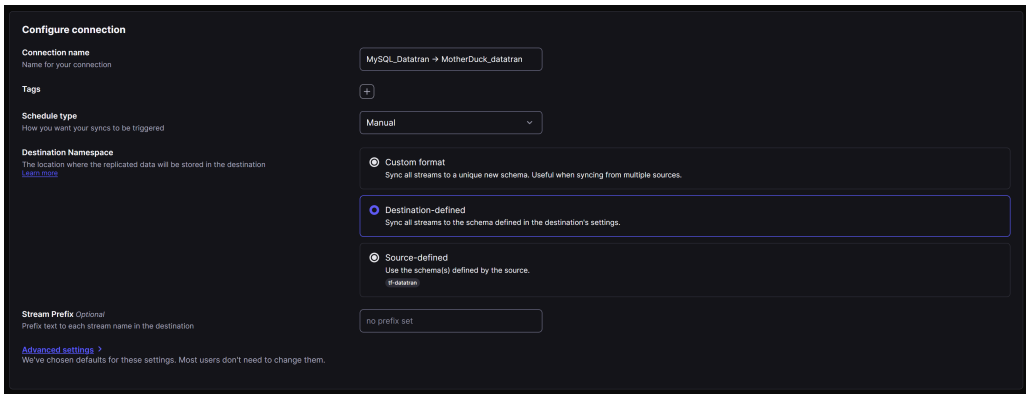

Figura 4: Configuración de la connection: `MySQL_Datatran → MotherDuck_datatran`, schedule Manual, namespace Destination-defined

1.4.5. Resultado del sync

El primer sync completó exitosamente en 2 minutos 5 segundos, transfiriendo 7,39 MB con los 11.380 registros de `accidentes_raw`.

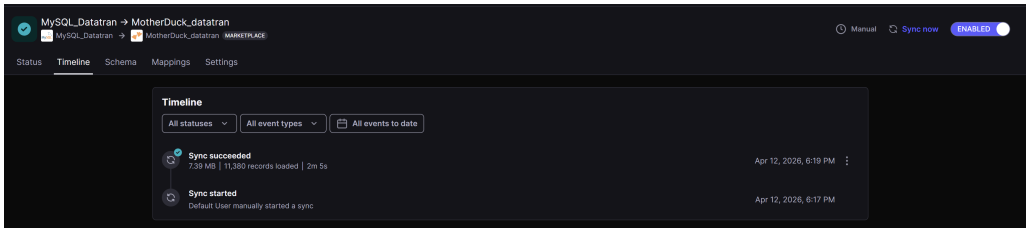

Figura 5: Timeline de Airbyte: sync exitoso — 11.380 registros cargados en MotherDuck

La verificación en MotherDuck confirma que los datos quedaron disponibles en `datatran.accidentes_raw` dentro de la base `airbyte_trabajo`:

Datatran

airbyte_trabajo

```
SELECT
  *
FROM datatran.accidentes_raw
```

11,380 rows returned in 2.6s

	br	id	km	uf	uop	ilesos	mortos	feridos	horario	pessoas	fase_dia	latitude	regi
1	153	742921	155	TO	UOP01-DEL02-TO	1	1	0	04:04:00	3	Amanhecer	-7,291548	SPRF-T
2	262	742942	146,1	MG	UOP03-DEL03-MG	1	0	4	06:40:00	6	Pleno dia	-20,0240733	SPRF-M
3	101	742943	193	SC	UOP01-DEL01-SC	1	0	1	06:58:00	3	Amanhecer	-27,48935222	SPRF-S
4	60	742947	23	DF	UOP01-DEL03-DF	1	0	1	07:05:00	3	Pleno dia	-15,988927	SPRF-D
5	163	742960	1044	MT	UOP03-DEL06-MT	1	0	1	06:17:00	3	Amanhecer	-10,14330603	SPRF-M
6	116	742971	568	SP	UOP02-DEL05-SP	0	0	1	08:50:00	2	Pleno dia	-25,05566	SPRF-SI
7	365	742985	393,4	MG	UOP01-DEL10-MG	1	0	2	12:00:00	3	Pleno dia	-18,63407	SPRF-M
8	40	742987	103,7	GO	UOP02-DEL01-DF	2	0	0	13:30:00	3	Pleno dia	-16,812437	SPRF-D
9	153	742988	245,5	MG	UOP02-DEL13-MG	1	0	0	09:30:00	2	Pleno dia	-20,294923	SPRF-M
10	262	742989	107,9	ES	UOP02-DEL01-ES	0	0	1	12:50:00	3	Pleno dia	-20,333803	SPRF-E
11	116	742993	386	BA	UOP01-DEL02-BA	1	0	0	10:35:00	4	Pleno dia	-11,904926	SPRF-B
12	101	743001	127	SC	UOP04-DEL04-SC	1	0	1	14:55:00	2	Pleno dia	-26,97015432	SPRF-S
13	364	743009	331,5	MT	UOP02-DEL05-MT	0	0	1	13:20:00	1	Pleno dia	-15,83158478	SPRF-M

Add Cell

Figura 6: Vista previa de `datatran.accidentes_raw` en MotherDuck — 11.380 filas en 2,6 s

1.5. Paso 4: Modelos dbt

El proyecto dbt en `workspaces/dbt_proyecto/` transforma los datos crudos en dos capas.

1.5.1. Configuración

```yaml
# profiles.yml
dbt_datatran:
  outputs:
    dev:
      type: duckdb
      path: "md:airbyte_curso"
      schema: datatran
      motherduck_token: "{{ env_var('MOTHERDUCK_TOKEN') }}"
  target: dev
```

```yaml
# dbt_project.yml
models:
  dbt_datatran:
    staging:
      +materialized: view
      +schema: staging
    marts:
      +materialized: table
      +schema: marts
```

1.5.2. Modelos staging

Los modelos staging aplican las transformaciones de tipo que no se pudieron hacer en MySQL:

Modelo	Transformaciones principales
`stg_accidentes`	`NULLIF(campo, 'NA')` para nulos literales; `REPLACE(latitude, ',', '.')::DOUBLE`; `REPLACE(km, ',', '.')::DECIMAL`; `CAST(id AS BIGINT)`, `CAST(mortos AS INT)` y demás enteros; `CAST(data_inversa AS DATE)`, `CAST(horario AS TIME)`
`stg_clima_openmeteo`	Tipado directo (ERA5 ya entrega datos numéricos); Decodificación de `weather_code` WMO a descripción legible; Join key: `accidente_id` (lat + lon + fecha + hora)

1.5.3. Modelo dimensional

Se implementa un esquema en estrella (Kimball) con las siguientes entidades:

Tabla	Descripción
`dim_fecha`	Fecha, día de la semana, fase del día, mes, año
`dim_ubicacion`	UF, municipio, BR, km, latitud, longitud
`dim_causa`	Causa del accidente, tipo, clasificación
`dim_via`	Tipo de pista, trazado, sentido, uso del suelo
`dim_clima`	Condición PRF + variables ERA5: precipitación, temperatura, código WMO
`fct_accidentes`	Tabla de hechos: víctimas, vehículos, FK a todas las dimensiones

La elección de Kimball sobre OBT se justifica por la presencia de dimensiones de alta cardinalidad (`dim_ubicacion` con 500 municipios, `dim_causa` con 40 causas distintas) que se benefician de la normalización para evitar redundancia y facilitar el filtrado en Metabase.

1.5.4. Tests de calidad (dbt-expectations)

Test	Modelo	Dimensión de calidad
`unique` + `not_null`	`fct_accidentes.id`	Unicidad / completitud

Test	Modelo	Dimensión de calidad
`expect_column_values_to_be_between`	`stg_accidentes.mortos`	Validez: rango 0–100
`expect_column_values_to_be_between`	`stg_accidentes.latitude`	Validez: rango −34 a 6
`expect_column_values_to_not_be_null`	`stg_accidentes.uf`	Completitud
`expect_column_proportion_of_unique_values`	`stg_accidentes.municipio`	Consistencia: >100 municipios
`expect_column_values_to_match_regex`	`stg_accidentes.data_inversa`	Formato YYYY-MM-DD

1.6. Paso 5: Orquestación con Prefect

El pipeline se orquesta con Prefect 3 desde `workspaces/prefect/pipeline.py`. Cubre las 7 etapas del pipeline de extremo a extremo, incluyendo la gestión de la infraestructura Docker y la verificación de datos antes de cada transformación.

1.6.1. Estructura del pipeline

```
@flow(name="datatran_pipeline", log_prints=True)
def datatran_pipeline():
    ensure_containers_up()     # docker compose up -d + espera MySQL
    verify_accidentes_raw()    # COUNT(*) en MySQL; corre initdb si es
necesario
    extract_openmeteo()        # extrae ERA5 y genera data/
clima_openmeteo.csv
    load_clima_raw()           # crea tabla + LOAD DATA INFILE en MySQL
    airbyte_sync()             # dispara sync y polling vía API REST
    dbt_run()                  # dbt deps + dbt run
    dbt_test()                 # dbt test
```

Task	Nombre	Descripción
1	`ensure_containers_up`	Ejecuta `docker compose up -d` y espera hasta que MySQL acepte conexiones (máx 60 s). Reintentos: 2.
2	`verify_accidentes_raw`	Verifica que `accidentes_raw` tiene datos. Si está vacía, ejecuta los scripts `01_schema.sql` y `02_load_data.sql` vía `docker exec` y confirma la carga.

Task	Nombre	Descripción
3	`extract_openmeteo`	Lee coordenadas únicas por fecha desde MySQL, las consulta en lotes de 100 a la API ERA5 de Open-Meteo y escribe `data/clima_openmeteo.csv`. Soporta reanudación. Timeout: 30 min.
4	`load_clima_raw`	Crea la tabla `clima_raw` en MySQL (si no existe) y ejecuta `LOAD DATA INFILE`. Saltea la carga si la tabla ya tiene filas.
5	`airbyte_sync`	Dispara el sync `MySQL_Datatran → MotherDuck_datatran` vía API REST y hace polling cada 10 s hasta completar o fallar. Timeout: 10 min.
6	`dbt_run`	Ejecuta `dbt deps` + `dbt run`. Reintentos: 1.
7	`dbt_test`	Ejecuta `dbt test` y falla el flow si algún test no pasa.

1.6.2. Configuración

```
cd workspaces/prefect
cp example.env .env     # completar MYSQL_PASSWORD y
AIRBYTE_CONNECTION_ID
pip install prefect mysql-connector-python
prefect server start &
python pipeline.py
```

La variable `AIRBYTE_CONNECTION_ID` se obtiene en Airbyte UI → Connections → Settings → Connection ID.

1.7. Paso 6: Visualización con Metabase

Metabase se levanta como contenedor Docker desde la misma imagen personalizada usada en las tareas anteriores, que combina Metabase v0.58.8 con el driver DuckDB de MotherDuck:

```
FROM eclipse-temurin:21-jre
ADD https://downloads.metabase.com/v0.58.8/metabase.jar /app/
metabase.jar
ADD https://github.com/motherduckdb/metabase_duckdb_driver/releases/\
download/1.4.3.1/duckdb.metabase-driver.jar /plugins/
```

1.7.1. Conexión a MotherDuck

En el setup inicial de Metabase se selecciona el driver **DuckDB** y se configura:

Campo	Valor
Nombre para mostrar	`Motherduck_Trabajo_Final`
Database file	`md:airbyte_trabajo`
MotherDuck Token	Token de la cuenta (campo separado del driver)
Use DuckDB old_implicit_casting option	Activado

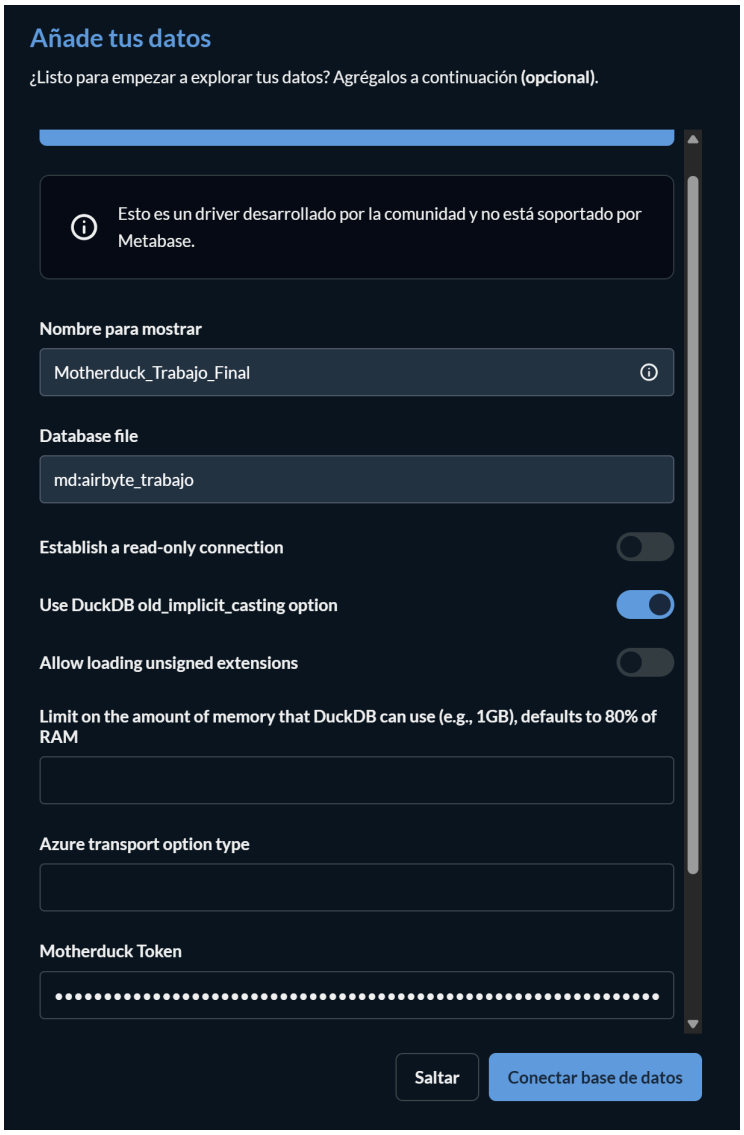

Figura 7: Configuración de la conexión DuckDB/MotherDuck en Metabase

1.7.2. Visualizaciones planeadas

#	Tipo	Pregunta de negocio
1	Mapa de calor	¿En qué estados y rutas se concentran los accidentes mortales?

#	Tipo	Pregunta de negocio
2	Línea temporal	¿Cómo evoluciona la cantidad de accidentes por mes?
3	Barra apilada	¿Qué causas predominan bajo precipitación vs cielo despejado?
4	Scatter plot	¿Existe correlación entre precipitación (mm/h) y mortalidad?
5	Tabla pivot	Mortalidad promedio por condición PRF vs código WMO ERA5

Filtros configurados: período de fecha, UF, causa del accidente, condición climática ERA5.

1.8. Conclusiones

El pipeline ELT implementado integra dos fuentes de datos heterogéneas — accidentes de tráfico en formato CSV con problemas de encoding y campos compuestos, y datos climáticos históricos de una API REST gratuita— en un modelo dimensional analítico en MotherDuck.

Las decisiones técnicas más relevantes fueron:

- **Encoding y pre-procesamiento fuera del contenedor:** la conversión Latin-1 → UTF-8 se realiza con un script Python antes de levantar MySQL, manteniendo la ingesta SQL simple y sin dependencias externas en el contenedor.
- **Todo TEXT en la capa raw:** evita que MySQL rechace o trunque datos con formatos no estándar (decimales con coma, nulos literales `NA`, fechas como string). dbt es el único responsable del tipado, lo que hace la transformación auditable y testeable.
- **Open-Meteo ERA5 sobre OpenWeather:** la API gratuita provee más variables relevantes para el análisis vial (ráfagas, nubes bajas, radiación solar) y no requiere gestión de claves API, simplificando la configuración y el despliegue.
- **Esquema estrella (Kimball):** la normalización en dimensiones facilita los filtros en Metabase y evita la redundancia que tendría una OBT con municipios y causas repetidos en cada fila de hecho.

Referencias

[1] Polícia Rodoviária Federal, «Dados Abertos de Acidentes de Trânsito nas Rodovias Federais». Accedido: 12 de abril de 2026. [En línea]. Dispo-

nible en: https://www.gov.br/prf/pt-br/acesso-a-informacao/dados-abertos/dados-abertos-acidentes

[2] P. Zippenfenig, «Open-Meteo.com Weather API». Accedido: 12 de abril de 2026. [En línea]. Disponible en: https://open-meteo.com/